

《漏洞利用及渗透测试基础》实验报告

姓名：王一诺 学号：2312331 班级：计科二班

实验名称：

堆溢出 Dword Shoot 攻击实验

实验要求：

以第四章示例 4-4 代码为准，在 VC IDE 中进行调试，观察堆管理结构，记录 Unlink 节点时的双向空闲链表的状态变化，了解堆溢出漏洞下的 Dword Shoot 攻击。

实验过程：

1. 创建项目与反汇编

打开 vmware 软件启动虚拟机打开 VC6, 新建一个项目输入实验源码。

```
#include<windows.h>
main()
{
    HLOCAL h1, h2,h3,h4,h5,h6;
    HANDLE hp;
    hp = HeapCreate(0,0x1000,0x10000); //创建自主管理的堆
    h1 = HeapAlloc(hp,HEAP_ZERO_MEMORY,8); //从堆里申请空间
    h2 = HeapAlloc(hp,HEAP_ZERO_MEMORY,8);
    h3 = HeapAlloc(hp,HEAP_ZERO_MEMORY,8);
    h4 = HeapAlloc(hp,HEAP_ZERO_MEMORY,8);
    h5 = HeapAlloc(hp,HEAP_ZERO_MEMORY,8);
    h6 = HeapAlloc(hp,HEAP_ZERO_MEMORY,8);

    _asm int 3 //手动增加int3中断指令，会让调试器在此处中断
    //依次释放奇数堆块，避免堆块合并
    HeapFree(hp,0,h1); //释放堆块
    HeapFree(hp,0,h3);
    HeapFree(hp,0,h5); //现在Freelist[2]有3个元素

    h1 = HeapAlloc(hp,HEAP_ZERO_MEMORY,8); |
    return 0;
}
```

可以看到，该程序利用 HeapCreate 函数创建了一个大小为 0x1000 的堆区，接着从堆中申请了六个堆块，每个堆块块身大小为 8 字节（加上块首 8 字节，实际大小应为 16 字节）。

随后是奇数堆块的释放，会有三个 16 字节堆块（h1、h3、h5）放入空表，也就是会依次放入 freelist[2] 所标识的空表。

后指令再次申请了 8 字节（加上块首为 16 字节）的堆区内存，也就是会从 freelist[2] 所标识的空表中取出第一个空闲堆块，而这正好是之前放入的 h1。因此只需要修改 h1 的前后指针就可以发生 Dword Shoot 攻击。接下来是对该过程中的内存变化的观察。

2. 内存具体变化观察

设置断点，F5 进入 Debug 调试，此时鼠标移动到 h1、h3、h5 等等上可以看到其地址。

Context: main[]	Address: 0x003a0680
Name	Value
h1	0x003a0688
hp	0x003a0000
Auto Locals this	
003A0680 04 00 00 00 44 07 18 00 00 00 00 00	
003A068C 00 00 00 00 AB AB AB AB AB AB AB AB	
003A0698 00 00 00 00 00 00 00 00 04 00 04 00	
003A06A4 40 07 18 00 00 00 00 00 00 00 00 00	
003A06B0 AB AB AB AB AB AB AB AB 00 00 00 00	
003A06BC 00 00 00 00 04 00 04 00 07 18 00 00	

可以看到 h1 的地址为 0x003a0688，再减去块首的 8 字节就可以得到块首的地址 0x003a0680，输入查看现在 h1 的内存状态。然后按下 F10 执行 h1 的释放，再观察该地址：

Address:	0x003a0680
003A0680	04 00 08 00 44 04 18 00 98 01 3A 00 98 01 3A 00
003A0690	EE FE EE FE EE FE EE FE EE FE EE FE EE FE EE FE
003A06A0	04 00 04 00 40 07 18 00 00 00 00 00 00 00 00 00
003A06B0	AB AB AB AB AB AB AB AB 00 00 00 00 00 00 00 00
003A06C0	04 00 04 00 4C 07 18 00 00 00 00 00 00 00 00 00
003A06D0	AB AB AB AB AB AB AB AB 00 00 00 00 00 00 00 00

可以看到其 flink（红框标出）和 blink（蓝框标出）都指向了 0x003a0198 同一个地址。根据堆块及空表的管理方式，这个地址应该就是 freelist[2] 的地址。输入该地址查看信息：

Address:	0x003a0198
003A0198	88 06 3A 00 88 06 3A 00 A0 01 3A 00 A0 01 3A 00
003A01A8	A8 01 3A 00 A8 01 3A 00 B0 01 3A 00 B0 01 3A 00
003A01B8	B8 01 3A 00 B8 01 3A 00 C0 01 3A 00 C0 01 3A 00
003A01C8	C8 01 3A 00 C8 01 3A 00 D0 01 3A 00 D0 01 3A 00
003A01D8	D8 01 3A 00 D8 01 3A 00 E0 01 3A 00 E0 01 3A 00
003A01E8	E8 01 3A 00 E8 01 3A 00 FA 01 3A 00 FA 01 3A 00

可以看到，freelist[2] 的 flink 和 blink 都指向了 0x003a0688 也就是 h1 的块身地址，这是因为当前 freelist[2] 这条链上只有 h1 一个空闲堆块链入了，二者的前向和后向指针都指向对方。

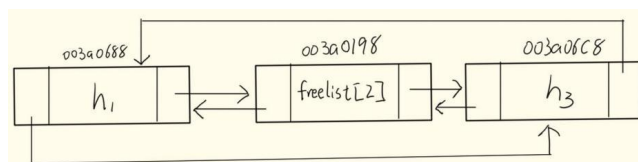
再按下 F10 执行 h3 的释放，释放后再查看 0x003a0198 也就是 freelist[2] 处的内存：

Context:	main[]	Address:	0x003a0198
Name	Value	003A0198	88 06 3A 00 C8 06 3A 00 A0 01 3A 00
h3	0x003a06c8	003A01A4	A0 01 3A 00 A8 01 3A 00 A8 01 3A 00
h5	0x003a0708	003A01B0	B0 01 3A 00 B0 01 3A 00 B8 01 3A 00
hp	0x003a0000	003A01BC	B8 01 3A 00 C0 01 3A 00 C0 01 3A 00
		003A01C8	C8 01 3A 00 C8 01 3A 00 D0 01 3A 00
		003A01D4	D0 01 3A 00 D8 01 3A 00 D8 01 3A 00

可以看到 freelist[2] 前向指针没变仍然指向 h1，而 blink 指向了 0x003a06C8，这是 h3 的块身地址，说明 freelist[2] 链入了一个新的空闲堆块（即 h3）。我们再输入查看 0x003a0680 也就是 h1 的内存状态是否有改变【下图左】：可以看到 h1 的 flink 指针发生变化，改为了指向 0x003a06C8 即 h3。

Address:	0x003a0680	Address:	0x003a06c8
003A0680	04 00 08 00 44 04 18 00 C8 06 3A 00	003A0698	EE FE EE FE EE FE EE FE
003A068C	98 01 3A 00 EE FE EE FE EE FE EE FE	003A06A8	00 00 00 00 00 00 00 00
003A0698	EE FE EE FE EE FE EE FE 04 00 04 00	003A06B8	00 00 00 00 00 00 00 00
003A06A4	40 07 18 00 00 00 00 00 00 00 00 00	003A06C8	98 01 3A 00 88 06 3A 00
		003A06D8	EE FE EE FE EE FE EE FE

再看看 0x003a06C8 即 h3 处的内存，可以看到 h3 前向指针指向 freelist[2]，后向指针指向 h1 块身。现在的空表结构大致为：



按下 F10 继续程序，执行 h5 的释放。完成后，再查看 0x003a0198 也就是 freelist[2] 处的内存：

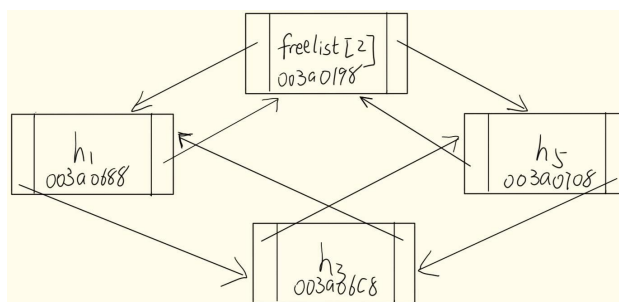
Context: main[]	Address: 0x003a0198
Name	Value
h1	0x003a0688
h5	0x003a0708
hp	0x003a0000
Auto	Locals
this	

003A0198	88 06 3A 00	08 07 3A 00	A0 01 3A 00
003A01A4	A0 01 3A 00	A8 01 3A 00	A8 01 3A 00
003A01B0	B0 01 3A 00	B0 01 3A 00	B8 01 3A 00
003A01BC	B8 01 3A 00	C0 01 3A 00	C0 01 3A 00
003A01C8	C8 01 3A 00	C8 01 3A 00	D0 01 3A 00
003A01D4	DA 01 3A 00	DA 01 3A 00	DA 01 3A 00

可见 freelist[2]前向指针没变仍然指向 h1，而 blink 指向了 0x003a0708，这是新链入的 h5 的块身地址。我们查询该地址看一下内存状态：

Address:	0x003a0708															
003A0708	98	01	3A	00	C8	06	3A	00	EE	FE	EE	FE	EE	FE	EE	FE
003A0718	EE	FE	EE	FE	EE	FE	EE	FE	04	00	04	00	70	07	18	00
003A0728	00	00	00	00	00	00	00	00	00	AB	AB	AB	AB	AB	AB	AB
003A0738	00	00	00	00	00	00	00	00	18	01	04	00	EE	14	EE	00
003A0748	78	01	3A	00	78	01	3A	00	EE	FE	EE	FE	EE	FE	EE	FE
003A0758	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF

可见 h5 flink 指针所指向的地址是 freelist[2]，后向指针 blink 指向的是 h3 块身的地址。再查看 h3（0x003a06C8）以及 h1（0x003a0688）的内存，可以大致画出现在的 freelist[2]空表结构：



最后我们重新分配一个加上块首有 16 字节的堆，这时候空闲表的第一个堆块也就是 h1 会被取下。这时我们查询 0x003a0198 即 freelist[2]处，可见 flink 变为了 0x003a06c8 即指向了 h3，即 h1 前向指针的值写入到了 h1 后向指针指向的内存里：

Address:	0x003a0198															
003A0198	C8	06	3A	00	08	07	3A	00	A0	01	3A	00	A0	01	3A	00
003A01A8	A8	01	3A	00	A8	01	3A	00	B0	01	3A	00	B0	01	3A	00
003A01B8	B8	01	3A	00	B8	01	3A	00	C0	01	3A	00	C0	01	3A	00
003A01C8	C8	01	3A	00	C8	01	3A	00	D0	01	3A	00	D0	01	3A	00

而且查询 0x003a06c8 也就是 h3 处，可见 h3 blink 指向了 0x003a0198 即 freelist[2]，即 h1 后向指针的值写入到了 h1 前向指针指向的内存里。

卸下一个堆块的时候，会将其前向指针和后向指针的值写入到其指向的内存当中，利用这个，便可以实现 Dword Shoot 攻击。

心得体会：

1. 通过 Dword Shoot 攻击的模拟和该过程中的堆块内存追踪，理解了 Dword Shoot 的攻击原理：用精心构造的数据去溢出下一个堆块的块首，改写块首中的前向指针 Flink 和后向指针 Blink，然后在分配、释放、合并操作发生时获得一次读写内存的机会。
2. 学会了如何申请堆块以及如何释放它们，以及如何追踪查看堆块的内存位置。更深入地了解了堆块和空闲堆表的管理方式，认识了 flink、blink 两个指针以及他们的变化。